

Demos Network © 2024 by Azhar Nurhussen, Cristiano Frassinetti, Jacob Ortiz Hansen is licensed under Attribution-NonCommercial-NoDerivatives 4.0 International

CC BY-NC-ND 4.0

Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

BY: Credit must be given to the creator(s).

NC: Only noncommercial use of this work is permitted. Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.

ND: No derivatives or adaptations of this work are permitted.

To view a copy of this license, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>

By reading, downloading, distributing, using and enjoying this work in any form, in whole or in part, you accept the above terms and conditions.

Demos: A BORDERLESS INTERCONNECTIVITY LAYER,
ENABLING SEAMLESS INTEROPERABILITY AND
ADVANCED FUNCTIONS ACROSS ALL CHAINS AND WEBS

Azhar Nurhussen | Cristiano Frassinetti | Jacob Ortiz Hansen
team@kynesys.xyz

Abstract

This paper presents a comprehensive analysis of Demos, a novel blockchain system that integrates various advanced concepts and technologies to create a secure, robust, efficient, scalable, and adaptable network. The design incorporates features such as Game Theory Incentives (GTI), Dynamic Validation Shard Size, and Shard Generation with a Common Validator Selection Algorithm (CVSA).

GTI is a unique incentivization strategy that promotes network stability, cooperation, and effective performance in a dynamic and decentralized gas-based blockchain. It utilizes game theory principles to encourage nodes to act in the best interest of the network. Nodes are assigned reputation scores based on their performance and contributions, which determine their likelihood of being selected as validators or proposers.

Dynamic Validation Shard Size enables the network to dynamically adjust the number of subdivisions (shards) based on transaction load and node distribution. This ensures optimal performance and resource utilization across varying network conditions. Load balancing techniques distribute the workload and nodes across shards, improving transaction processing speeds and reducing latency.

Shard Generation with CVSA ensures consistent and secure shard configuration by creating a shared, deterministic source of randomness. The process involves initiating an epoch, applying the CVSA, verifying and distributing the output, generating shards based on the output, and assigning nodes to these shards. This approach enhances security, scalability, and fairness in the network.

The paper also addresses other important aspects, including on-chain Web2 integration and cross-chain logic execution.

On-chain Web2 integration allows native access to Web2 resources within the blockchain network, leveraging cryptographic algorithms for authenticity and integrity verification. Crosschain logic execution enables scripting, execution, and consolidation of smart contracts and transactions across multiple networks, combining their strengths and mitigating limitations.

Through the integration of these advanced concepts and technologies, the proposed blockchain system design demonstrates the potential for creating a highly secure, scalable, and efficient network suitable for a wide range of applications.

It offers innovative solutions to incentivize node participation, adapt to varying network conditions, ensure secure communication, and provide efficient storage capabilities in a (highly) decentralized manner.

Table of Contents

License-----	1
Introduction-----	5
Demos Core Structure and Principles-----	6
Overall Architecture-----	8
Nodes and Clients-----	8
Nodes and Validators-----	8
The Role of Demos-----	9
A Trustless Data Layer: towards the Omniweb-----	9
An intent-based environment-----	10
-----	11
demosWork: the core structure for Omniweb communications-----	12
Gas Tanks: Demos Metafees State-----	13
Demos Architecture & Consensus Mechanism-----	15
Decentralized Adaptive Architecture (DAA)-----	15
Overview-----	15
Mechanism-----	15
1. Global Layer Status (GLS)-----	15
2. Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT)-----	15
3. Game Theory Incentives (GTI)-----	15
4. Dynamic Validation Shard Size-----	16
Advantages-----	16
The Global Layer Status (GLS)-----	17
Mechanism-----	17
Key Features and Solutions-----	17
Advantages-----	17
Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT)-----	19
The Two Consensus Tiers-----	19
1. Byzantine Fault Tolerance (BFT) within a Shard-----	19
2. Proof-of-Representation (PoR) for a Representative Shard-----	20
Advantages-----	20
Game Theory Incentives (GTI)-----	21
Design and Mechanism-----	21
1. Dynamic Gas Fees-----	21
2. User Agnostic Profit Flexibility-----	21
3. Competitive Landscapes-----	21
Advantages-----	21
Shard Generation and Validators Choice with a Common Validator Selection Algorithm (CVSA)-----	22
Phase 1 - Initiate CVSA Prerequisites-----	22
Phase 2 - Apply CVSA-----	22
Phase 3 - Verification and Distribution-----	22
Phase 4 - Shard Generation-----	22
Phase 5 - Begin Epoch Operations-----	23
Dynamic Validation Shard Size-----	23
Design and Mechanism-----	23
Recap-----	24
The Importance of Being Futureproof: Post Quantum Cryptography-----	25
The Demos PCQ Stack-----	26
The Rijdael algorithm-----	26
The Argon2 algorithm-----	27
The McEliece algorithm-----	27
The Dilithium algorithm-----	28
Demos Modularity-----	29
On-chain Web2-----	30
Design and Mechanism-----	30
Securing Web2 Context: Data Agnostic HTTP(S) Relay Proxies-----	30
Ensuring Integrity and Stability-----	33
Request Throttling-----	33
End-to-End Validation-----	33
Session Based Approach-----	34
Towards the Fediverse: ActivityPub integration-----	35

TOR Network: A Special Context Made for Privacy -----	38
Demos Network and TOR: A Natural Synergy-----	38
Crosschain Logic Execution-----	39
Design and Principles-----	39
Securing XM Context-----	40
Read, Write, and Crosschain Presence-----	40
Web2 and Cross-Web Logic-----	42
Example: Crosschain Swapping-----	42
Design and Multiple Routes-----	42
Demos Firewall-----	43
Constant Auditing-----	43
Immediate Response Propagation-----	43
L2 Parallel Subnetwork (L2PS)-----	45
What is a Subnetwork?-----	45
Why Parallel?-----	45
IMP: Demos built-in L2PS-----	46
Privacy and L2PS: A built-in concept-----	47
The ZK (Zero Knowledge) Module-----	48
The Importance of a ZK Module in an Omnichain-----	48
Demos and Zero Knowledge-----	48
Demos ZK Module and beyond-----	49
Homomorphic Encryption: Tailoring Privacy for Real Use Cases-----	50
A Practical Example of FHE-----	50
A Trustless Approach to Data Transformation-----	50

License

Demos: A BORDERLESS INTERCONNECTIVITY LAYER, ENABLING SEAMLESS INTEROPERABILITY AND ADVANCED FUNCTIONS ACROSS ALL CHAINS AND WEBS

© 2023 by Azhar Nurhussen, Cristiano Frassinetti, Jacob Ortiz Hansen is licensed under CC BY-NC-ND 4.0

1. **1. CC BY-NC-ND 4.0**

2. Attribution-NonCommercial-NoDerivatives 4.0 International

This license requires that reusers give credit to the creator. It allows reusers to copy and distribute the material in any medium or format in unadapted form and for noncommercial purposes only.

- **BY:** Credit must be given to the creator.
- **NC:** Only noncommercial use of this work is permitted. Noncommercial means not primarily intended for or directed towards commercial advantage or monetary compensation.
- **ND:** No derivatives or adaptations of this work are permitted.

To view a full copy of this license, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>

Introduction

The following document presents Demos, an innovative decentralized network that interconnects and expands existing blockchain technology to address the limitations and challenges faced by traditional blockchain systems. Demos introduces novel concepts and mechanisms that aim to revolutionize the way decentralized networks operate and deliver trust, transparency, and scalability.

This yellow paper elucidates the key principles, design, and functionality of Demos, aiming to provide a comprehensive understanding of its inner workings. By exploring the decentralized nature of Demos and its nodal composition, we delve into the intricate mechanics that underpin this groundbreaking network.

Demos Core Structure and Principles

Demos represents a decentralized blockchain system, encompassing a distributed framework consisting of interconnected nodes.

Demos is based on the Demos Network specifications, SDK and framework.

As a fully self-contained blockchain, built from the ground up and designed to facilitate the creation of an omniweb, Demos includes several features and modules.

This paper reflects the current state of the Demos Network specifications as of 18/01/2024.

In the following text, those features will be discussed in detail.

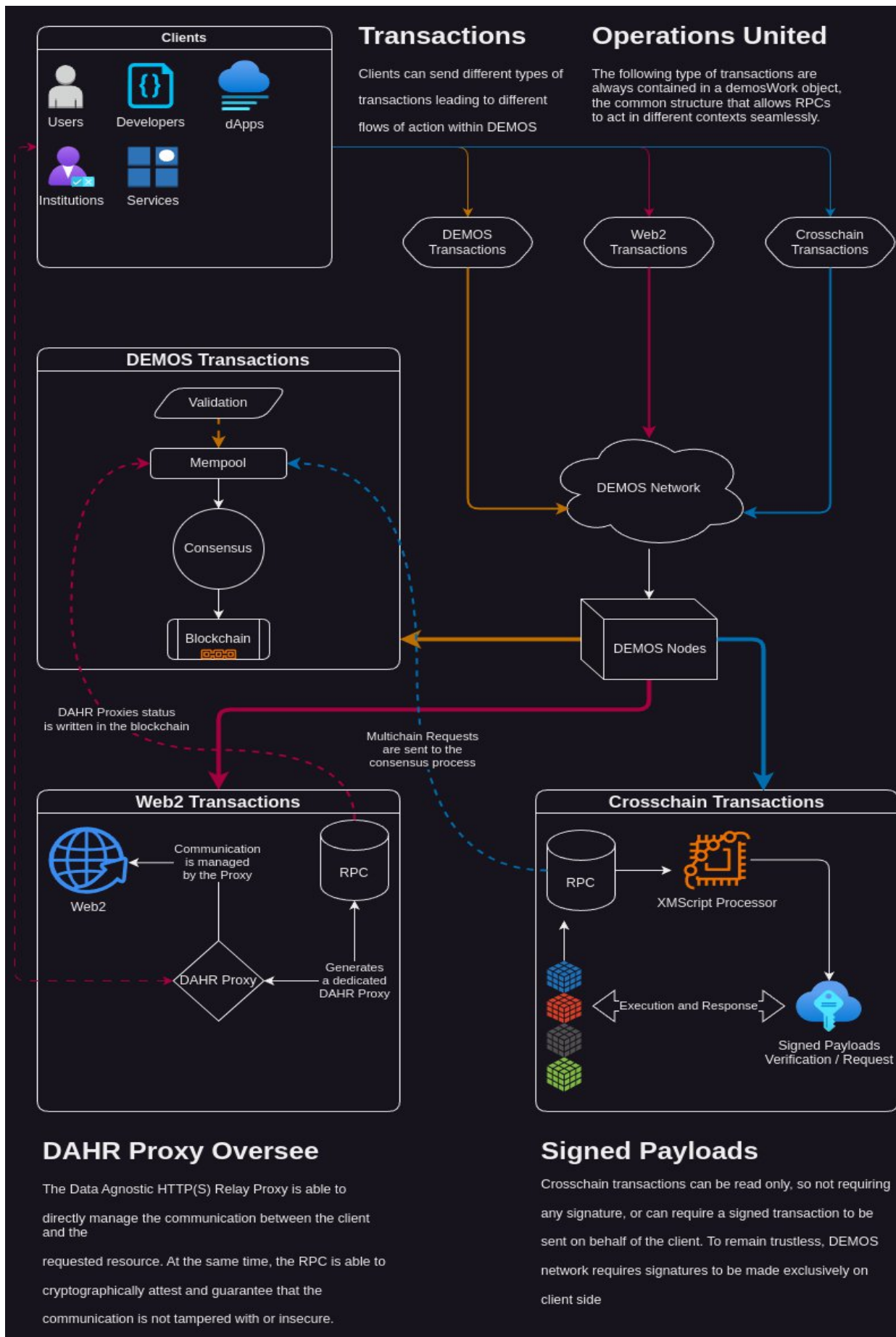


Figure 1: Demos Network Architecture

Overall Architecture

According to Figure 1 above, Demos is able to accept and process different types of transactions coming from and going to different contexts through its network of nodes.

A context is defined as the domain in which a data source can be found. For example, to retrieve a resource hosted on a website, the context is Web2.

The two main contexts in which Demos operates are Web2 and XM (Crosschain Media). For each of them, Demos defines a strict set of rules and workflows to ensure integrity and security.

Those rules will be discussed in the following paragraphs.

Nodes and Clients

The term "node" refers to an entity that follows the Demos specifications by providing a client/server architecture that is compatible with other nodes of the same network.

Given the public nature of Demos specifications, it is both feasible and strongly recommended to develop Demos clients capable of actively engaging in the Demos network. This approach fosters client diversity and enhances the overall functionality of the network.

Nodes and Validators

The Demos architecture is designed to enforce a possible validator role for every node within the Demos network. As a result, the Demos network achieves a decentralized blockchain structure, leveraging a particular consensus mechanism that heavily relies on the resilience and security principles inherent in the blockchain paradigm.

In order to ensure the commitment of validators and mitigate the risk of majority attacks, the operation of a node is contingent upon meeting specific prerequisites akin to staking (for further details, please refer to the next section). This approach aims to incentivize and reward honest and productive participants within the network while penalizing those who engage in malicious activities or exhibit inadequate performance.

The Role of Demos

In order to grasp the essence of Demos' purpose and its potential capabilities, it is imperative to gain a comprehensive understanding of the network's fundamental role, delving into the intricacies of its multifaceted offerings.

Demos, functioning as a network-agnostic omniweb infrastructure, has been meticulously architected to facilitate seamless communication, interaction, and mediation among the realms of web2, web3, and the myriad of existing blockchains.

While Demos acknowledges and recognizes the inherent value and benefits offered by diverse L1, L2, and L3 blockchains currently in existence, its primary objective is not to undermine their utility. Rather, Demos assumes the role of decentralizing the blockchains themselves, empowering users and developers to harness the optimal attributes embedded within each distinct chain.

As explained in this paper, all the features that Demos provides are made with the specific goal of enhancing decentralization without reinventing the wheel. Each technology and blockchain possesses a unique set of strengths and weaknesses.

It is within this context that Demos emerges as a solution, a data layer that enables seamless interoperability among different technologies. Moreover, Demos enriches the entire ecosystem by incorporating its own distinctive features, thereby fostering a cohesive and unified environment.

A Trustless Data Layer: towards the Omniweb

While being suitable for a vast array of operations, the main Demos goal is to establish itself as a Data Layer, or Data Facilitator. The core philosophy of Demos is that there are great projects and working infrastructures that suffer from being isolated from each other, with the best case being a sturdy, non standard compatibility.

We call all of these isolated systems “contexts”. If we see each one of these contexts (such as blockchains, web2 protocols and so on) as railways, we see a huge number of working railways whose trains (data) can't travel on the others. Demos is a meta-railway: the railway that connects and makes compatible the other ones using a common interface.

An intent-based environment

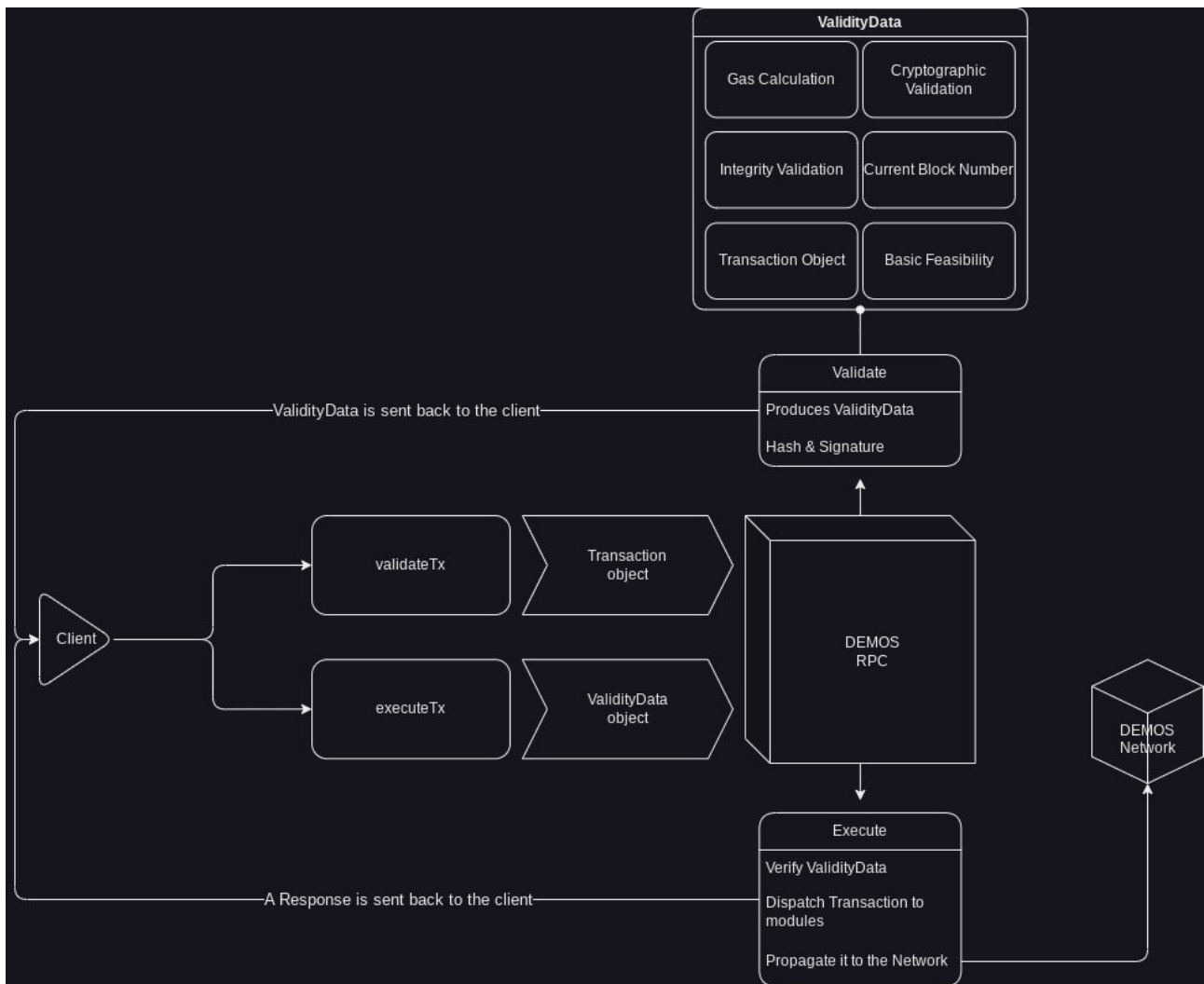
The Demos network is designed to be **intent-centric**. While the term can be confusing as it applies to many possible mechanisms, it is very easy to explain how the Demos blockchain manages intents.

Each transaction, as shown in the schematic below, contains information that **assigns an intent** to a client with **no degree of uncertainty**. This means that the client expresses its will to perform the operations contained in the transaction at a given timestamp, with a given target block and in a reproducible manner.

Thanks to this, Demos Network is able to **assign a precise chronological order** to its shared mempool: **frontrunners and prority fees are not supported** in this scenario, so that once the transaction has been correctly executed by the blockchain, finality can be guaranteed **even before the next block is mined**.

In this mechanism, the inclusion of a transaction in a block is an **acknowledgement of the integrity** of the data and the retrieval process that led to the execution of the included payloads.

Each transaction is essentially an **intent** that, once resolved by the network through the mechanisms explained in this paper, is binding and already executed. It should be noted that Demos Network acts as a **supervisor** of the execution of intents: for this specific reason, it guarantees impartiality and certainty even within the same block.



An example of how Transactions are managed within the Demos Network

demosWork: the core structure for Omniweb communications

As mentioned in the previous paragraph, the goal of Demos is to **enable efficient, readable and extensible** communication between **current and future contexts**. Therefore, Demos has been designed to provide a future-proof and feature-rich interface for this communication.

Each Demos transaction contains an object called **demosWork**, which is exposed in the Demos SDK and in the specifications. This object defines a **set of operations** that can be Web2, Crosschain, TOR or any other context specific and supports conditional behaviour.

In this way, dApps and clients in general will be able to **bundle** several different operations into a single transaction, with a common class that allows both developers, users and node software to work in a **harmonious** way, while maintaining **readability and simplicity**.

A demosWork is basically a **JSON structure**, so it can either be written manually or generated by the SDK. This means that the generated demosWork acts as a **scripting language** that tells the RPC what to do and how to do it within the demos specifications.

Gas Tanks: Demos Metafees State

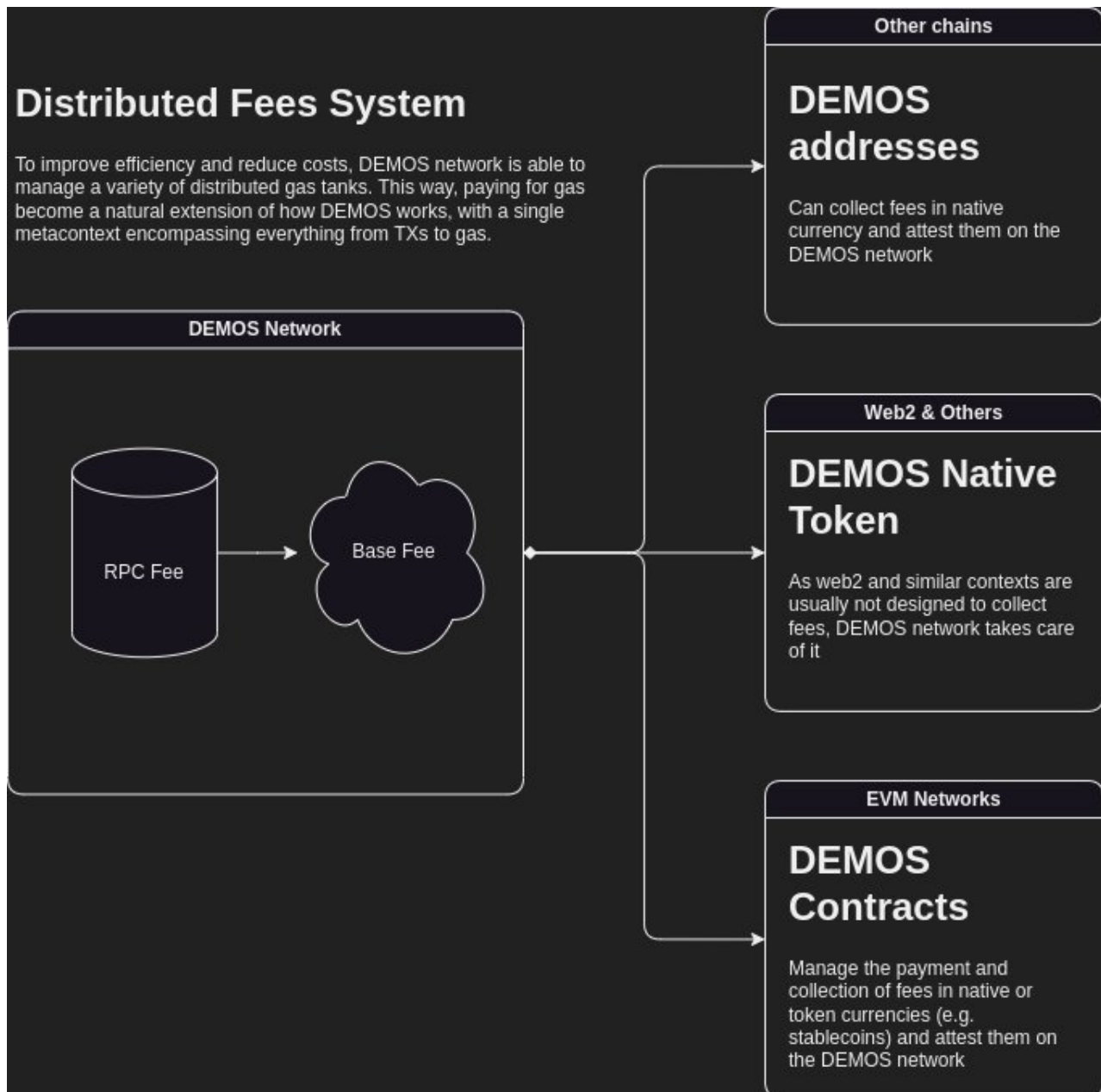


Figure1: Fees management allows Demos to build a metafee state

The Demos charging mechanism, as explained in the relevant section, allows customers and nodes to retain flexibility without sacrificing sustainability. However, things change when dealing with several different networks.

Being isolated and self-sufficient, each blockchain requires fees in its own currency (with some exceptions). What's more, it's very likely that clients won't have every single native coin for the chains they want to operate on.

If a client wants to do an XM transaction with writes on, say, chain A, B and C, they would normally have to provide A tokens, B tokens and C tokens for gas, plus the Demos network fees.

This option is inconvenient and unacceptable. In order to harmonize gas requirements, Demos creates entities called Gas Tanks on different networks. Depending on the network architecture, a Gas Tank can be a smart contract, a wallet, a chainware or any other compatible form.

The role of the Gas Tanks is to always be in real-time contact with the Demos Network and to collect fees in the chain's native currency or (if supported) in the chain's specific token. The Demos Network allows developers to specify how they wish to pay for gas, so that a developer can quickly execute their transaction with the tokens they hold.

In the background, each gas tank is known to the network and provides its capacity so that the Demos Network always knows the total gas balance and how it is distributed.

An example might be as follows

- Alice sends an XM Transaction involving a write operation on a EVM network and two L1s
- Demos Network calculate the max fees requested in the various steps and presents Alice with coherent options to pay gas
- Alice, holding just the EVM network native token, pays with that network token
- The EVM Network gas tank is filled and synced, while the RPC converts (if needed, usually done in advance) the needed tokens

This way, Alice just has to hold a single token instead of 3 different ones.

Demos Architecture & Consensus Mechanism

Decentralized Adaptive Architecture (DAA)

Overview

Decentralized Adaptive Architecture (DAA) is an innovative architecture model designed to achieve exceptional efficiency, scalability, and performance in decentralized systems while operating as a parachain infrastructure. DAA leverages dynamic system adaptation, game theory, and representative sharding technology to ensure robust security.

Mechanism

1. Global Layer Status (GLS)

To enhance scalability and efficiency, DAA implements the Global Layer Status (GLS) paradigm. This mechanism involves processing transactions in the most decentralized and collaborative way so that it is easy, fast, and efficient to find and scrap invalid or malicious payloads. Moreover, the Global Layer Status ensures consistency across shards while allowing them to operate independently.

2. Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT)

DAA employs a two-layer byzantine consensus mechanism, combining Proof-of-Representation (PoR) with Byzantine Fault Tolerance (BFT) for updating the Global Layer Status. Nodes are selected to form a Representative Shard for the validation process based on the PoR algorithm that ensures a deterministic non-tamperable selection method. This incentivizes stability and efficiency. Within the Representative Shard, a Byzantine Fault Tolerant consensus protocol is implemented, guaranteeing robustness, security, and scalability without burdening the entire chain.

3. Game Theory Incentives (GTI)

To help ensure profitability, competitiveness, and commitment, DAA incentivizes nodes via a free market fee system. Using game theory principles, nodes are able to provide their services for a certain fee over the base chain fee, so that each node competes with the others to offer the most efficient and cheapest service possible while maintaining profitability.

4. Dynamic Validation Shard Size

DAA continuously adapts to varying transaction volumes by dynamically adjusting the number of participants to the block proposing consensus phase. This adaptability is achieved through a randomness layer powered by a Common Validator Selection Algorithm, ensuring the generation of a dynamic and consistent Representative Shard.

Advantages

1. **Enhanced Scalability:** Through the implementation of the Common Validator Selection Algorithm, the Decentralized Adaptive Architecture (DAA) system achieves superior processing capabilities by concurrently handling a multitude of transactions within a deterministic representation of all the nodes. This substantially augments the network's overall capacity, enabling seamless scaling.
2. **Robust Security:** The DAA system incorporates the classic Byzantine Fault Tolerant (BFT) consensus technique in conjunction with the Proof-of-Representation model to establish a formidable security framework that effectively mitigates a wide range of potential attack vectors. This amalgamation of methodologies ensures the integrity and resilience of the system against malicious activities.
3. **Heightened Efficiency:** DAA demonstrates remarkable efficiency by dynamically adjusting the proposers' count in response to the network's load. Leveraging the power of BFT, independent cross-node validation is facilitated, leading to optimized processing speeds and resource utilization. The system operates at peak efficiency, bolstering overall performance.
4. **Sustained Decentralization:** By integrating Game Theory Incentives, DAA upholds the fundamental principles of decentralization within the blockchain ecosystem expanding it to include a fair and competitive fee system. This mechanism incentivizes nodes to collaborate in the best interest of the entire network, fostering a robust and resilient decentralized environment. The preservation of decentralization ensures equitable participation and governance within the DAA system.

The Global Layer Status (GLS)

GLS is a mechanism that allows transactions to be executed with the possibility of rolling back, examining, and verifying their content even before a block is emitted.

Mechanism

1. A transaction reaches an RPC and is validated for identity and integrity. The signature and the hashes are examined and confirmed, then the transaction is sent to the GLS Preprocessor.
2. Global Layer Status (GLS) Preprocessor: The GLS functions as a comprehensive ledger that maintains the state of the entire blockchain network. The Preprocessor examines the transaction and builds a delta (a sorted registry of changes) to the Global Layer Status without modifying it directly.
3. Each subsequent transaction follows the same process as above. Only when a block needs to be forged from the mempool, all of the GLS Preprocessor registry items are executed one by one in the right order on a copy of the GLS.
4. If a transaction produces an invalid result in the GLS, is rejected or reverted. Otherwise, the GLS gets permanently modified once the block is emitted by the consensus mechanism.

Key Features and Solutions

1. Predictive Transaction Validation

Thanks to the local GLS change registry, each node is able to predict with reasonable precision if a transaction will fail or will succeed. Even before the next block is emitted, many elements and mechanisms allow a transaction to be simulated with efficiency and rapidity.

2. Resource-Friendly Rollbacks

For the same reason, as the GLS change registry is not enforced until the block is emitted, it is very easy to track and roll back invalid, malicious, or malformed transactions. Even if a transaction makes it before the GLS changes registry, once the consensus validator parses it and flags it as invalid there is no need to do anything else other than scrapping the transaction from the registry.

Advantages

1. Scalability

GLS (Global State Ledger) leverages parallel processing of transactions within individual nodes and employs efficient global state management techniques to significantly enhance the network's capacity.

2. Efficiency

The employed strategy effectively minimizes cross-node communication overhead and optimizes GLS updates, thereby achieving a notably high level of operational efficiency.

3. Consistency

By diligently maintaining a synchronized state of the blockchain across all nodes, GLS ensures a unanimous agreement among all network nodes regarding the current network state.

4. Enhanced Interoperability

The incorporation of GLS in the system facilitates improved interoperability among different nodes within the blockchain network. Consequently, any node gains the ability to seamlessly access the state of any address present across the entire network.

Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT)

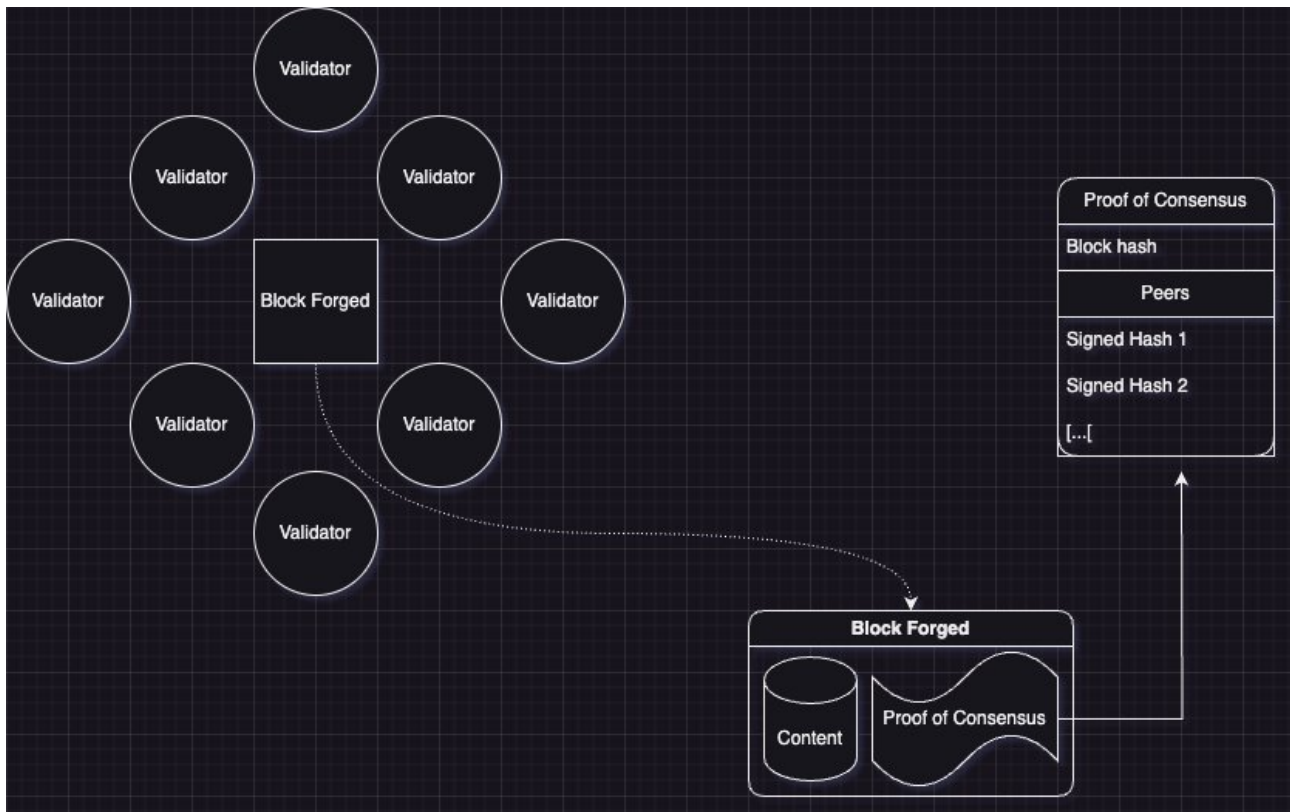


Figure2: The emerging properties of the consensus mechanism can be seen as a Proof of Consensus

Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT) is an advanced consensus mechanism that operates at a heightened level of technical sophistication, effectively ensuring network stability, efficiency, and security within a blockchain system.

It accomplishes this through the integration of a Byzantine Fault Tolerance (BFT) consensus mechanism within a shard representing the entire validators network, selected with the utilization of Proof-of-Representation (PoR) for the proposal of blocks to the Global Layer Status (GLS).

This shard is called the Representative Shard and is composed of the result of an algorithm that, using the deterministic property of the transactions in the mempool, ensures the same group of validators are chosen by all the legitimate nodes without any need of data transfer.

The Two Consensus Tiers

1. Byzantine Fault Tolerance (BFT) within a Shard

The PoR-BFT system leverages the Byzantine Fault Tolerance consensus mechanism. BFT enables rapid validation and agreement of transactions within the shard, even when confronted with the presence of malicious or faulty nodes. By mandating the agreement of a supermajority (typically $2/3+1$) of nodes prior to appending a new block to the shard's blockchain, BFT ensures

efficient operation and robust security within a group of validators.

2. Proof-of-Representation (PoR) for a Representative Shard

To elect the Representative Shard, the PoR-BFT system employs the PoR mechanism. Validators selected for this process are chosen based on deterministic values contained in the GLS and in the intrinsic blockchain properties, which will always be identical for all the nodes that act on the same, untampered and verified data. This way, malicious nodes or malfunctioning nodes are automatically excluded from the consensus selection without any network overhead.

The exact mechanism that allows this to happen is described in the '[Shard Generation and Validators Choice with a Common Validator Selection Algorithm \(CVSA\)](#)' paragraph.

Advantages

1. Efficiency

The utilization of a Byzantine Fault Tolerant (BFT) consensus mechanism within a Representative Shard facilitates rapid transaction processing and streamlined intra-shard operations.

2. Augmented Security

The Proof of Representation (PoR) consensus mechanism significantly strengthens security measures during the validators' election process to the next consensus round. By selecting nodes using a deterministic approach based on data integrity, PoR effectively mitigates the risks associated with malicious activities, fostering heightened integrity and efficiency within the blockchain system.

3. Scalability

The PoR-BFT approach offers an advanced and scalable solution that can effectively manage a substantial volume of transactions. This approach ensures consistency and reliability within the GLS, even in the face of high transaction loads, thereby enabling the system to seamlessly handle increased demands without compromising performance. This is done by changing the size of the Representative Shard to facilitate computation and data transfer even in case of massive traffic.

4. Stability

The PoR mechanism promotes stability within the blockchain network by avoiding data overhead and by incentivizing long-term participation from network nodes. By dynamically selecting a Representative Shard for the whole blockchain, PoR fosters an environment of reliability and predictability, ensuring the overall stability of the blockchain system.

Game Theory Incentives (GTI)

Game Theory Incentives (GTI) is a unique incentivization strategy that promotes network stability, cooperation, and effective performance in an extremely low gas fee blockchain system. Rather than relying on direct financial rewards, GTI utilizes principles from game theory to encourage nodes to act in the best interest of the network.

Design and Mechanism

1. Dynamic Gas Fees

Each node within the system calculates the base network fee to apply to a transaction using a GLS shared state and a common algorithm. The Demos network expects this fee to be compliant and will require that amount to be paid fully. Factors as network load, complexity and size all concur to define how much is this fee.

2. User Agnostic Profit Flexibility

On top of the base Demos fee, nodes are able to request an additional fee which quantity is arbitrarily decided by the node maintainer. This way, node operators can define a profit that is independent from the user and that will make running a node profitable.

3. Competitive Landscapes

This possibility enables a competitive landscape where each node has to find the right balance between offering competitive fees and having enough profit to be efficient and performant. As per the game theory, a landscape like this brings to the decentralization of the actual gas fee based on the node attractiveness for a developer or an user.

Advantages

1. Encourages Active Participation

GTI encourages nodes to maintain active and reliable participation in the network, as this increases their chances of being selected as RPC by developers and users.

2. Enhances Security

By rewarding efficient and transparent nodes, GTI helps maintain the integrity and security of the network.

3. Promotes Fairness

The natural selection process ensures all nodes, regardless of their age, have a chance of being an RPC.

4. Improves Network Performance

GTI encourages cooperation among nodes, leading to improve overall network performance.

Shard Generation and Validators Choice with a Common Validator Selection Algorithm (CVSA)

The process of shard generation using a Common Validator Selection Algorithm (CVSA) involves creating a shared, deterministic source of randomness to ensure all nodes generate the same shard configuration given the same network state.

Phase 1 - Initiate CVSA Prerequisites

At the commencement of each validation round, the blockchain system initiates the CVSA process. This process commences by selecting an initial input, commonly denoted as the "seed." The determination of this seed is derived from various immutable aspects of the network itself in the following way:

- The 'lastBlockHash' variable is filled with the last block's hash
- The 'nextBlockNumber' variable is filled with the next block's number
- The 'hashedStakes' variable is filled by hashing all the stakes that are currently expressed in the GLS
- The 'baseSeed' is manipulated so that it includes the shard size in its value and becomes the 'seed'

Thanks to the above verifiable variables, a 'baseSeed' is generated and used for the following steps.

Phase 2 - Apply CVSA

The selected seed is then input into a pseudo-random number generator to generate a CVSA. The CVSA will be identical for those nodes compliant with the blockchain mechanism while will be different for malicious or malfunctioning nodes. The output remains impervious to prediction until the computation is finalized and needs to be demonstrated by cryptography, ensuring security and unpredictability.

Phase 3 - Verification and Distribution

Once the CVSA has generated the output, all other nodes will possess the same information without the need for transmission. The inherent property of rapid verification in the CVSA guarantees simultaneous propagation of the output to all nodes.

Phase 4 - Shard Generation

Each node in the network uses this output as the seed for shard generation and validation assignment. Since the seed is identical for all nodes, the shard generation process results in the same set of shards across the entire network. The nodes are then assigned to these shards based on the deterministic process using the same seed, ensuring uniform distribution and a balanced

workload.

Phase 5 - Begin Epoch Operations

With the shards generated and nodes assigned, the blockchain network proceeds with its operations within the new epoch, processing transactions within each shard, and committing states to the Global Layer Status.

The utilization of a CVSA within the randomness layer provides a secure, verifiable source of randomness, enabling consistent shard generation across the network, enhancing security, and ensuring scalability.

Dynamic Validation Shard Size

Dynamic Validation Shard Size entails the implementation of a sophisticated approach wherein the number of actors chosen between the nodes network to operate the consensus, known as Representative Shard, within the blockchain network can be dynamically adjusted in response to variations in transaction load and node distribution. This adaptability grants the blockchain network the ability to uphold optimal performance levels across a diverse range of conditions, spanning from scenarios of low to high transactional demand.

Design and Mechanism

1. Choosing a Representative Shard: each block time, the Demos blockchain utilizes a deterministic algorithm that returns the same exact result on all the nodes that have the same data in their mempool and in their blockchain. This way, all the nodes are able to instantly know who the participants are without any communication needed.
2. Parameters: to create the Representative Shard, parameters based also on the chain load, throughput, and specific conditions are enforced to determine the size of the Representative Shard.

Recap

The newly proposed blockchain consensus mechanism presents a robust, scalable, and innovative solution, integrating features of Global Layer Status (GLS), Proof-of-Representation with Byzantine Fault Tolerance (PoR-BFT), Game Theory Incentives (GTI), Dynamic Validation Shard Size, and a secure Randomness Layer powered by a Common Validator Selection Algorithm (CVSA).

Through ACS-GLS, the system efficiently manages transaction processing and data storage across various shards, enhancing scalability while ensuring a consistent view of the global state. The PoR-BFT consensus model integrates a Byzantine Fault Tolerance consensus within each shard with a Proof-of-Representation mechanism for global block proposals, optimizing both intra-shard and inter-shard operations.

The GTI framework encourages active, reliable participation and cooperative behavior among nodes, fostering network stability and security while ensuring profitability and fairness. Dynamic Validation Shard Size enhances the system's adaptability, ensuring optimal performance and resource utilization regardless of network load. This, combined with the integration of a CVSA-powered Randomness Layer, ensures secure, verifiable, and consistent shard generation, further contributing to the network's robustness and security.

This innovative approach to generating shared randomness enables the system to maintain uniformity and fairness in the node distribution and workload allocation, further enhancing the network's scalability.

This novel blockchain system design showcases the potential of integrating various advanced concepts and technologies in building a blockchain network that is not only secure, robust, and efficient but also highly scalable and adaptable to varying network conditions.

The Importance of Being Futureproof: Post Quantum Cryptography

Blockchains depend heavily on cryptography to ensure a robust level of security while handling highly sensitive information. For an algorithm to be considered secure, it must stand the test of time, a criterion met by widely used algorithms such as ED25519 and RSA.

These algorithms are particularly favored for their implementation in user-friendly environments. However, the ability to withstand the test of time is not infinite for ED25519 and RSA, and this limitation applies to blockchains that utilize different, often weaker, algorithms as well.

The emerging field of quantum computing poses a significant challenge. Quantum processors, while not yet consumer-grade, have already shown substantial improvements in efficiency and computational power. The advent of quantum computing becoming accessible to the general public, or at least to a well-connected subset, is not far off.

So, what is the solution if even the most secure algorithms may eventually fail? The National Institute of Standards and Technology (NIST) is actively encouraging the development of quantum-resistant cryptography. They regularly publish a list of recommended algorithms for public study.

Demos, designed to withstand this issue, is capable of switching its cryptography and hashing modules to entirely new algorithms upon consensus. Post Quantum Cryptography modules are already integrated into the Demos source code, awaiting activation or updates. While we are actively testing and verifying highly regarded algorithms such as McEliece or Lattice-based ones, the Demos network is designed to continuously update its "backup" algorithms.

Demos Network specifies and ships additional cryptography modules exposing the exact same interfaces as the default ones.

By default, Demos Network utilizes ED25519 and RSA for identity management and encryption. Those algorithms are more than battle proven but they are not guaranteed against Quantum attacks.

That's why Demos specifications are constantly updated to reflect the cutting edge algorithms available for implementation. If, in any moment, the Network should be in need of using those algorithms, through the usual consensus they can be used as drop-in replacements without any downtime or compatibility issue

Post-Quantum Cryptography (PQC) Module is perhaps the best example of how the mentioned modularity can benefit a blockchain and allows us to delve deeper into this aspect of the blockchain.

The Demos PCQ Stack

Demos Network specifications, while being highly dynamic due to the active ongoing research in the PCQ field, exposes in particular four modules designed to offer flexibility and a reasonable working environment in case of drop-in replacement.

The four algorithms, chosen based on the NIST recommendations, are:

- Rijndael; symmetric encryption algorithm considered the state of the art of its category
- Argon2; quantum-safe hashing algorithm
- McEliece; post-quantum cryptography algorithm that uses a keypair to share secrets between two parties.
- Dilithium; post-quantum cryptography algorithm that uses a keypair to sign and verify messages.

The Rijndael algorithm

The Rijndael algorithm, which underpins the Advanced Encryption Standard (AES), is a symmetric key encryption algorithm renowned for its quantum resistance and is often preferred over AES-256 for several reasons. While AES-256 is a specific implementation of Rijndael with a fixed block size of 128 bits and key sizes of 128, 192, or 256 bits, Rijndael itself offers greater flexibility with variable block and key sizes that can be independently specified to any multiple of 32 bits, ranging from 128 to 256 bits.

This flexibility allows for the use of larger block sizes, potentially enhancing security against certain types of cryptographic attacks. In the context of blockchain and post-quantum cryptography (PQC), the robustness of the Rijndael algorithm is particularly significant. Blockchain technology relies heavily on cryptographic principles for security, and with the advent of quantum computing, there is a growing need for quantum-resistant cryptographic algorithms.

Rijndael's design, which was chosen for its resistance to all known attacks, source code compactness, and speed across various computing platforms, makes it an attractive option for blockchain applications that require high levels of security and efficiency. Moreover, the performance of Rijndael is noteworthy; it has been shown to be faster than AES on certain platforms, especially when considering the setup time for larger keys.

Its simple algorithm and implementation characteristics contribute to its efficiency and flexibility, which are crucial for the high-throughput and low-latency requirements of blockchain networks.

In summary, the Rijndael algorithm's variable block and key sizes, coupled with its performance advantages, make it a preferable choice for blockchain applications that demand robust security and high-performance encryption.

While AES-256 is considered secure and quantum-safe, Rijndael's design principles and adaptability provide additional benefits that are essential for the future-proofing of blockchain technology against quantum threats.

The Argon2 algorithm

The Argon2 algorithm, heralded as the winner of the 2015 Password Hashing Competition, represents a significant advancement in cryptographic hashing, particularly for data security.

Designed by experts from the University of Luxembourg, Argon2 is engineered to provide robust defense against a spectrum of dehashing attacks, setting a new standard in hashing technology.

Unlike its predecessors, such as SHA-1 and SHA-256, which are vulnerable to brute-force attacks due to their relatively faster computation times, Argon2 introduces key features that significantly bolster its security posture.

In comparison to SHA-1 and SHA-256, Argon2's design philosophy emphasizes not just resistance to brute-force attacks but also protection against more sophisticated threats, such as side-channel and TMTO attacks. Its memory-hardness, combined with the ability to adjust critical parameters, provides a more nuanced and effective approach to data hashing, making it a superior choice for safeguarding sensitive information in an increasingly security-conscious digital landscape as the Omniweb.

The McEliece algorithm

The McEliece algorithm is used to encrypt and decrypt messages, much like a symmetric classic encryption algorithm. Thanks to its post-quantum security, however, it is not possible to retrieve the secrets as easily as with a classic algorithm.

The algorithm's immunity to quantum attacks is primarily due to its resistance to Fourier sampling and coset state measurements, techniques that quantum computers could potentially use to break encryption.

Furthermore, the McEliece algorithm has been shown to require an increase in key sizes by a factor of four to counteract improvements in quantum information set decoding, highlighting its adaptability to evolving quantum computational capabilities.

We use McEliece to exchange a long-term secret between two parties. This secret will be the base to generate one-time secrets encrypted with McEliece itself that will be used to generate one-time symmetric keys.

In the context of blockchain technology, where security and efficiency are paramount, the McEliece algorithm's post-quantum security, fast encryption and decryption, and resistance to quantum attacks make it an attractive option for securing long-term secrets and generating one-time symmetric keys for transactions. Its ability to exchange long-term secrets between parties securely, which then serve as the basis for generating one-time secrets encrypted with McEliece itself,

underscores its potential to safeguard blockchain transactions against both current and future cryptographic threats

The Dilithium algorithm

The Dilithium algorithm emerges as a cutting-edge solution in the realm of digital signatures, particularly within the blockchain ecosystem, where the integrity and authenticity of transactions are paramount. Unlike traditional signature algorithms such as ed25519, Dilithium not only offers post-quantum security—making it resilient against the potential future threat posed by quantum computing—but also introduces the capability to generate combined signed messages. This innovative feature allows for the verification of signatures without the necessity to disclose the original message, thereby providing an additional layer of privacy and security.

In the blockchain context, where transactions and their associated data must be securely and efficiently validated, Dilithium's post-quantum security ensures long-term resilience against quantum attacks. This is crucial as the advent of quantum computing could render many current cryptographic methods vulnerable. Dilithium's security foundation is built on the hardness of lattice problems, specifically the Short Integer Solution (SIS) and Learning With Errors (LWE) problems, which are considered intractable even for quantum computers.

This positions Dilithium as a robust safeguard for blockchain technologies, ensuring that digital signatures remain secure in the face of evolving computational capabilities. Moreover, the ability of Dilithium to produce combined signed messages that facilitate signature verification without revealing the initial message is particularly beneficial for blockchain applications. This feature enhances privacy and security, allowing for the authentication of transactions in a manner that conceals the transaction's details from third parties. Such a capability is invaluable in scenarios where confidentiality and data integrity are of utmost importance.

Furthermore, Dilithium's performance and efficiency make it well-suited for the high-throughput and low-latency demands of blockchain networks. Its design, which avoids the complexities of Gaussian sampling in favor of uniform sampling, simplifies implementation and reduces the risk of side-channel attacks, thereby ensuring both security and practicality in deployment.

In summary, the Dilithium algorithm stands out as a superior choice for blockchain applications, offering post-quantum security, the innovative feature of combined signed messages for enhanced privacy, and a design optimized for efficiency and ease of implementation. Its robustness against quantum threats, coupled with its practical benefits, makes it an essential tool for securing the future of blockchain technology.

Demos Modularity

As a highly dynamic blockchain with a growing number of integrations, features and contexts, the Demos network has been designed with modularity in mind since day 0.

Modularity is a broad concept and is often used as a synonym for "extensibility". While modules can certainly extend the capabilities of a network, it wouldn't be fair to exclude the other side of the coin: maintainability.

In the following paragraphs we will examine both as they are defined in the Demos network specifications.

Extensibility

The first consequence of modularity is of course extensibility. This means that every Demos client has to follow common and standard practices defined by the Network so that it can quickly update its features if requested.

The Demos Network is able to, through consensus, insert or remove certain features. In that case, the consensus outcome must be followed so that the new features are both implemented and tested

Maintainability

The second but not less important consequence of modularity is maintainability. As stated above, the Demos Network is able to quickly propagate updates in a safe way, leveraging the blockchain's consensus and traceability so that both patches and rollbacks are possible without hindering the network security..

On-chain Web2

The establishment of a seamless linkage between web2 and web3 serves as a crucial solution to a significant impediment hindering the emergence and flourishing of many projects and technologies.

Demos Network leverages its decentralized structure and battle-proven cryptographic algorithms to provide seamless, native, and trustless access to web2 resources from within blockchains.

Design and Mechanism

At the core of web2 integration, there is a specific endpoint available to dApps and clients that allows actors to ask for a specific resource such as an API, a GraphQL request or a plain HTTP(S) session. Once the RPC receives a correctly compiled request on the web2 endpoint, it first validates the request by checking the hash of the request and its signature to ensure authenticity and integrity.

It then creates an HTTP(S) proxy between the client and the requested resource, allowing direct communication between the two, while being able to monitor the communication process to ensure that no tampering or malicious actions have taken place.

While the communication is in progress, the RPC can attest to its validity, integrity and legitimacy without having to decrypt the data it contains. In this way, even if a request is encrypted using HTTPS, the RPC is able to maintain the security of the communication channel without compromising the user's privacy.

The employed hashing algorithm used to guarantee the cryptographical security of the communication channel is SHA256, and the signature method utilized is ed25519. This empowers the on-chain retrieval and provision of web2 data without reliance on external oracles or centralized solutions.

While this mechanism is secure enough to be trusted, a couple of control mechanisms are in place to also ensure performance, error handling, and the resolution of specific exceptions that may occur during the integration of web2 and web3.

Securing Web2 Context: Data Agnostic HTTP(S) Relay Proxies

To ensure a verifiable, integer and trustless way of accessing Web2 data on demand, Demos network follows strict specifications to be able to build, spawn and control a chain of trust.

This means that a Web2 transaction follows a very rigorous workflow that is designed to avoid both data tampering and uncertainty.

To better illustrate the above explanation, let's examine the typical flow of a Web2 transaction:

- The client produces a valid Web2 transaction and sends it to the chosen Demos RPC
- The Demos RPC generate a dedicated DAHR (**Data Agnostic HTTP(S) Relay**) Proxy

- The DAHR proxy is presented to the client, which uses it to communicate natively with the requested resource.
- The client is now able to process and maintain proper communication with the Web2 resource without the classic limitations of an on-chain oracle.
- At the same time, the RPCs monitor this communication channel by observing and certifying it without interfering
- In this way, the client not only receives the resource immediately, guaranteeing immediate finality, but is also always aware of the integrity of the communication.
- When the next block is mined, the current attestation data and status for that DAHR proxy will be added to the blockchain.

This way, while allowing customizations in terms of speed and security, Demos is able to guarantee integrity and verifiability of the data provided.

The novel approach to Web2 data pulling on chain exemplifies the need of a paradigm shift from a push-oriented, controlled and limited offering of oracles to the pull-oriented, on demand and self-verifiable offering of data feeds.

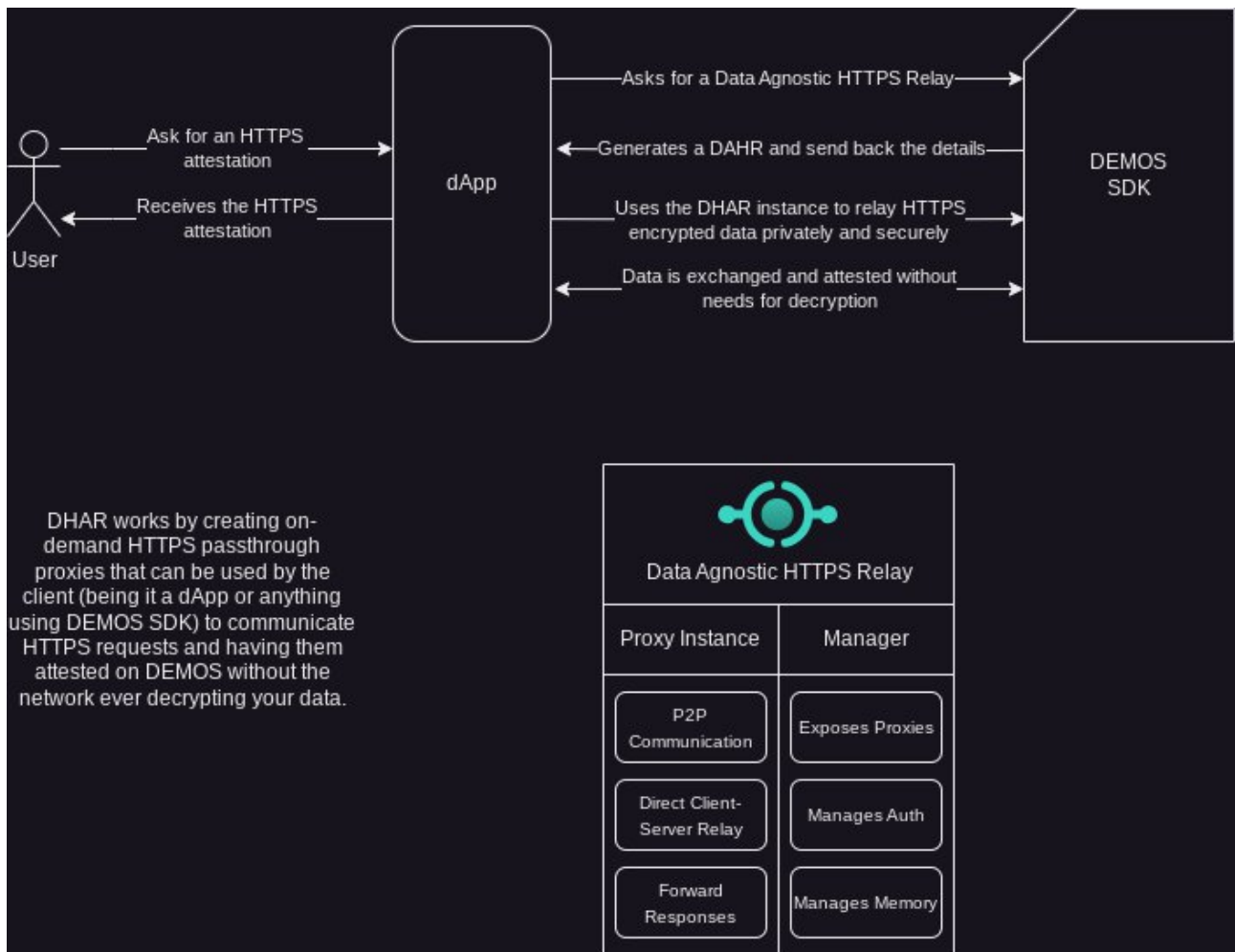


Figure 3 Architecture of a DAHR Proxy

Ensuring Integrity and Stability

To mitigate the unpredictable nature of web2 data retrieval and prevent potential clogging and security concerns, a robust system must be put in place. This system employs a multitude of techniques such as request throttling, End-To-End validation, and replication verification to ensure optimal performance and security.

Request Throttling

To prevent overloading low-performance endpoints and to ensure that responses are received within an acceptable timeframe, request throttling is employed. This technique involves limiting the number of requests that can be made within a specific timeframe. By implementing request throttling, web2 systems effectively manage their resources, preventing congestion and optimizing performance.

Request throttling is implemented using various techniques built on two main paradigms: rate limiters, which restrict the number of requests based on a predetermined rate, and timeframe restriction, which restricts the timeframe in which a request can be retrieved, attested, and sent back.

End-to-End Validation

End-to-End (E2E) validation constitutes a crucial technique for guaranteeing the authenticity of web2 data retrieval. This process involves thorough verification of the identities of all parts involved before any data exchange takes place. By enforcing E2E validation, web2 systems effectively counter potential security threats, such as man-in-the-middle attacks and unauthorized access.

On the web3 side of the request, E2E validation is implemented using on-chain signature-based identity verification, while digital certificates are employed on the web2 side to ensure the integrity of the request and response flow.

Session Based Approach

Derived from the DAHR Proxies design, Web2 Retrieval on the Demos chain supports and encourages a session-based approach. This means that a client is able to negotiate a fixed, prepaid gas charge with the RPC so that it is able to access data in a continuous way, for example by creating a streaming session with the resource (such as WebSockets), without having to pay a fee each time the resource is refreshed.

Not only does this help to keep costs at an acceptable level for both the client and the RPC, but it also drastically reduces the time required to retrieve a resource as the gas transactions are not necessarily made each time.

Towards the Fediverse: ActivityPub integration

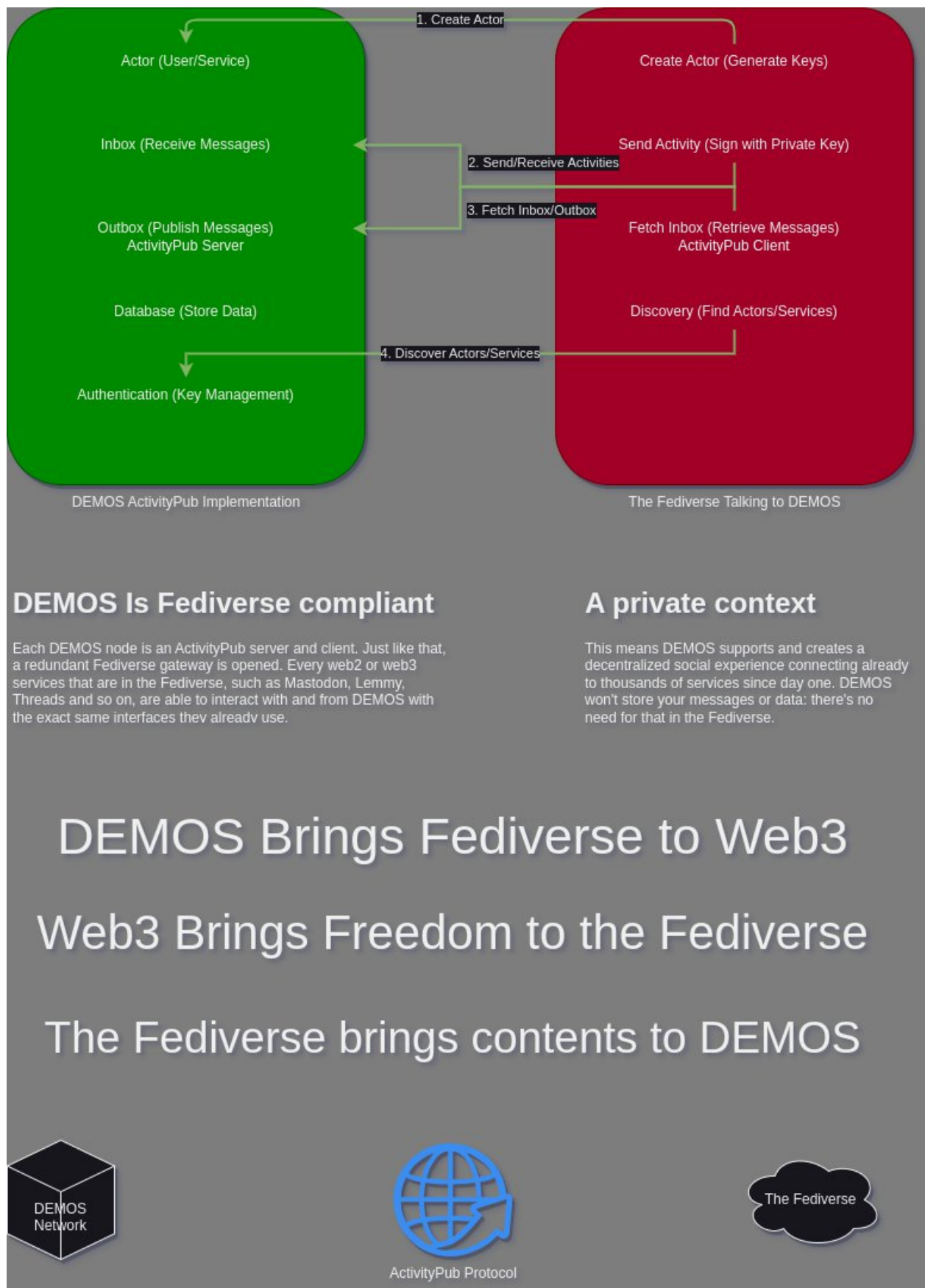


Figure 4: Federating through Demos is as easy as it can get

As Demos strives to leverage existing solutions expanding their horizons, the Fediverse is a natural target for the Network.

The Fediverse is a network of connected social services that implements the ActivityPub protocol. By doing this, millions of users on thousands of services are able to interact between each other no matter which platform they are using.

The most famous examples of ActivityPub social networks are Threads by Meta, Mastodon, and Lemmy.

By incorporating ActivityPub on-chain, Demos is not just connecting social networks but is also streamlining the exchange of data between any services utilizing the protocol. This integration is particularly impactful because it taps into an existing ecosystem of ActivityPub-compatible services, ranging from content management systems like WordPress to specialized platforms for podcasting, link aggregation, and more.

The ability to transmit data across these services without the need for additional bridging software or custom integrations represents a significant leap forward in interoperability. Moreover, the on-chain implementation of ActivityPub by Demos ensures that data transmissions are secure, transparent, and immutable, leveraging the strengths of blockchain technology. This approach not only facilitates data sharing and collaboration across various platforms but also enhances user control over their data, as they can trust the integrity of the data being transmitted on the blockchain.

In essence, Demos is not just integrating a social networking protocol; it is embedding a universal data transmission standard into its infrastructure. This positions Demos as a key player in the Fediverse, capable of driving forward the vision of a truly interconnected and decentralized web of services. The implications of this are vast, as it opens up possibilities for a more fluid exchange of information and a more cohesive digital ecosystem where data can flow freely between services that were previously siloed.

In a practical scenario, consider a user posting an update on Threads, a social network. While Threads is integrated with Instagram, the reach of the user's post is typically confined to these platforms. With Demos's ActivityPub integration, this post can be transmitted across a myriad of ActivityPub-compatible services, including those in the Fediverse like Mastodon, Pleroma, PeerTube, and Diaspora, as well as Web3 platforms, protocols, chains or any other context.

By definition, however, ActivityPub is designed for Web2 and has no native Web3 capabilities.

Demos, as a Data Interconnection Layer, implements in its specifications an ActivityPub compliant module: this way, different clients both in Web2 and Web3 are able to interact through the Fediverse and federate their instances.

Let's look at an example:

- Alice is on Threads and posts something about her garden
- Bob is on a RandomSocial, a fictional social network fully on chain
- RandomSocial is connected with Demos and thus can provide Bob with Alice's content seamlessly

- At the same time, Bob is able to use RandomSocial to reply to Alice without leaving the Web3 context

Being channeled through the existing security features of Demos Network, this data flow is verifiable, confidential (if requested) and consistent.

This integration not only enhances the interoperability among different social networks and Web3 platforms but also ensures secure, transparent, and immutable data transmissions, thanks to the inherent properties of blockchain technology. As a result, Demos is opening up the Fediverse to Web3, creating a more interconnected and expansive digital ecosystem.

TOR Network: A Special Context Made for Privacy

As a blockchain prioritizing privacy, Demos Network specifications establish a compatibility layer with TOR (The Onion Router).

TOR, a privacy-centric alternative network, interconnects various nodes to guarantee the anonymity of its network members. To provide a simplified insight into TOR's operation, its Onion architecture, reflected in its name, relies on the direct connection of multiple layers, each aware only of the preceding and succeeding relay.

This approach can be viewed as "Security through Ignorance," aligning seamlessly with the philosophy of Demos Network.

Demos Network and TOR: A Natural Synergy

Demos specifications establish a standardized and universal approach to accessing Web2 data. From a technical standpoint, Demos is inherently compatible with TOR when appropriately configured. The specifications include a mandatory compatibility layer, enabling clients and RPCs to utilize the TOR network for requests that would typically occur on the clearnet.

Despite the inevitable increase in delay and finality, inherent to the security of the TOR protocol, the Demos Network adeptly manages tasks such as .onion links resolution, hidden services provisioning, and overall TOR-based privacy communication, particularly in the Web2 and IMP context.

Notably, Demos specifications define the TOR compatibility layer as a module that must seamlessly integrate with any other Demos module. To illustrate a potential use case, consider a Demos transaction conducted through TOR in a Web2 scenario:

Here is an example of a possible use of Demos through a TOR Web2 Transaction:

- Alice, residing in a censored country, wishes to access the Unbiased News Journal.
- Locally, Alice creates a Demos Transaction, requesting the Web2 resource over TOR.
- Alice transmits the encrypted Demos transaction.
- The Demos RPC decrypts the transaction and initializes the Web2 module with TOR.
- Upon retrieving the clearnet or hidden service data, the result is encrypted and sent back to Alice.
- Only Alice possesses the capability to decrypt the message at her discretion.
- The only data retained by the RPC Node is the attestation of the request.

Crosschain Logic Execution

The concept of Crosschain Execution is the integration of diverse domains, combining attributes like security and speed by empowering developers to script, execute, and consolidate smart contracts and transactions across multiple networks.

By harnessing the strengths of individual networks and mitigating their respective limitations, Crosschain Execution establishes a more efficient and secure system as a whole. For instance, developers can leverage the speed and efficiency of one network while compensating for its lack of robust security features by utilizing another network that offers enhanced security capabilities.

This approach offers flexibility and adaptability, enabling developers to select the most suitable network for each specific task, unrestricted by a single network. It fosters a decentralized ecosystem, devoid of any dominant controlling entity, by creating an interconnected layer with Demos, facilitating seamless data and functional transfer between networks. This interconnectedness allows for efficient resource exchange and utilization, promoting a cohesive and synergistic ecosystem.

Design and Principles

Cross-chain Execution refers to a novel design principle that facilitates efficient and rapid interaction between different blockchains. This advanced concept is particularly advantageous in scenarios where smart contracts or a collection thereof necessitate a substantial computational workload.

For instance, consider a dApp developer who desires to maintain the security of a gas-intensive chain like Ethereum while capitalizing on the lower gas costs associated with a less robust chain such as Binance Smart Chain (BSC). The introduction of Cross-chain Execution has made it possible to execute portions of a smart contract on BSC, while the ultimate outcome is validated and finalized on Ethereum. By adopting this approach, developers can harness the swiftness and efficiency of BSC while simultaneously upholding the security and functionality offered by Ethereum.

Furthermore, Cross-chain Execution empowers developers to create intricate dApps that exploit the unique strengths of multiple blockchain networks, without confining themselves to a single chain.

To enable the implementation of Cross-chain Execution, Demos has developed user-friendly standardized endpoints and software development kit (SDK) functions. These resources facilitate cross-chain execution for both clients and developers. The platform's unified architecture adheres to the principle of interoperability, ensuring seamless collaboration among diverse blockchain networks.

Each supported chain encapsulates its available methods, such as establishing a connection with a provider or retrieving balances, within a standardized object that exposes identical methods across all supported chains. This standardized approach simplifies the execution of transactions across different blockchain networks, as developers can utilize a single method, such as

"Connect," which remains applicable to every supported chain.

Moreover, the platform's SDK functions furnish developers with a straightforward and intuitive interface to interact with the blockchain network, thereby streamlining the process of building decentralized applications.

The SDK undergoes continuous maintenance and updates to accommodate the expanding range of supported blockchains. Importantly, these enhancements are implemented without necessitating developers and users to make significant changes to their existing code, thereby minimizing the learning curve while preserving flexibility and simplicity.

Similarly, transaction execution adheres to the same underlying principles, bolstered by an additional layer of security that mandates authentication of the requester before executing any write-like operation.

Securing XM Context

Crosschain data transfer and thus interoperability is a very delicate field. Among the myriad of possible security problems, the main barrier for developers and clients is the non-existent standardization of chain operations.

Demos implements the Demos SDK XM module, as depicted in the schema above. With the SDK, clients can seamlessly construct multichain transactions in an intuitive manner or through a straightforward scripting language known as XM Script. The Demos RPC processes the XM Script, receiving it and returning the results along with attestation data from all the involved chains.

While the aim of this process is to be as swift as possible, it's crucial to acknowledge that the speed of a transaction is limited by the slowest network within the operation. This holds particularly true when each transaction requires finality. However, for read-only transactions, this logic generally doesn't apply as they typically don't necessitate finality.

Read, Write, and Crosschain Presence

To enable interoperability across various blockchains, Demos employs a comprehensive framework that supports two distinct types of operations within the domain of Crosschain Execution.

The Read operations entail retrieving data from target networks without requiring explicit authentication, except for specific scenarios. For instance, examining the account balance on Solana does not necessitate possessing a Solana account, and this principle holds true for nearly all read operations across diverse networks. This operational category proves immensely valuable for swiftly obtaining data without the need to engage in time-consuming processes like handshake establishment, authentication, and data propagation associated with on-chain modifications.

The Write operations demand meticulous and trusted authentication on the target network to ensure secure execution. Since these operations involve altering data on-chain, utmost caution, efficiency, and discretion must be exercised to ensure data accuracy and identity integrity. By leveraging client-side signatures when possible, Demos effectively serves as a virtual RPC mechanism for the target network, while concurrently exposing the aforementioned universal

Demos methods. Consequently, Demos does not require the user's private key or sensitive information, instead acting solely on behalf of the requester for that specific request, utilizing the specific content provided by the user.

On the other hand, certain operations may necessitate a more direct approach wherein relayed transactions relying solely on signatures prove insufficient. In such scenarios, Demos assumes an intermediary role, executing logic directly on a blockchain while soliciting additional requirements from the requester, such as gas fees or specific information.

In both cases, Demos is able to select the most efficient route and deliver an intuitive, user-friendly, and privacy-focused solution that remains agnostic to the underlying blockchain, benefiting developers, users, and clients alike.

XM Security on Write transactions

It's crucial to highlight a key aspect of the Demos SDK XM Module implementation: by default, the module doesn't transmit, demand, or in any way request sensitive information.

For Write transactions, whether they are EVM-related or involve transfers on other Layer 1 networks, the framework equips clients with all the essential methods to generate and sign transactions and payloads entirely on the client side. With the exception of the "Execute" step, which necessitates reachability to the RPC, it is entirely feasible to construct a fully compliant XMScript/XM Transaction offline.

This methodology is referred to as "Security through Ignorance" and forms the foundation of the Demos Network's communication approach.

Here is an example of a possible XM Transaction workflow:

- The client initiates script editing entirely offline by disconnecting their router.
- Upon realizing the need to use the Pay method on the EXAMPLE (XMPL) chain, the client utilizes the SDK to locally sign that operation.
- Operations, both read and write, are continuously added to the script.
- Upon completion, the client possesses an object (a XMScript) containing all operations in their specified order.
- Write operations include signed payloads devoid of sensitive information.
- The client reconnects the router and dispatches the request to Demos RPC.
- Demos RPC employs the Demos XM Module to execute, relay, and manage the included operations without any modification.
- The resulting data is interpreted and processed, ensuring the client receives it in a readable format.

This way, no sensitive data can be leaked in any part of the process.

Web2 and Cross-Web Logic

As Demos is built around different features that can and will interact with each other, one of the derived functionalities, known as Cross-web Execution, is a combination of Crosschain Execution and On-chain Web2, with its technical explanation being the sum of these constituent elements. It is necessary to highlight this synergistic feature, which may not be immediately evident.

By employing the established methodologies for on-chain retrieval of web2 resources with assured integrity and security, alongside seamless and secure crosschain smart contract execution, the convergence of these approaches gives rise to unparalleled capability.

For instance, a dApp can incorporate smart contract logic deployed on an EVM network while leveraging attested web2 data as input. Conversely, developers have the flexibility to feed the results of a smart contract into a web2 API, such as an AI endpoint.

The potential applications are virtually limitless, contingent upon the performance characteristics of the involved endpoints. Consequently, the resulting data can be once again utilized in a similar manner across other networks, endpoints, or technological frameworks.

Example: Crosschain Swapping

Crosschain Swapping, as an implementation of cross-chain execution, constitutes a distinctive use case that necessitates a dedicated exposition of its fundamental mechanics.

To facilitate a comprehensive understanding of cross-chain swapping's underlying technical principles, it is beneficial to provide a brief explication of this feature. Crosschain Swapping entails the seamless, rapid, and efficient execution of direct swaps between disparate blockchain networks. For illustrative purposes, we shall consider a practical scenario where an ERC20 token is acquired using a Stablecoin on the Solana blockchain.

Design and Multiple Routes

During the development of cross-chain swapping, it became evident that a single method could not consistently deliver the optimal combination of efficiency, speed, and security.

Due to varying conditions on both sides of the swap operation and within the Demos network itself, Demos must possess the intelligence to select the best overall route from a set of possible options.

To address this requirement, we have devised two primary scenarios in which Demos can make decisions regarding the chosen path.

1. Full Liquidity Availability:

In the ideal situation, Demos will possess sufficient liquidity on both sides of the swap, enabling it to promptly provide liquidity after validating the swap request. In this case, Demos will either distribute the requested token directly to the buyer's address or, if necessary, exchange the available liquidity for the requested token before distribution. This method, which is the

fastest and preferred approach, will be employed whenever the Full Liquidity Availability condition is met.

2. Limited Liquidity Availability:

When Demos lacks the necessary liquidity on both sides of the swap, it acts as a bridge and ramps aggregator with enhanced security measures. This scenario is expected to occur during the initial stages of Demos implementation.

While maintaining the same interface and syntax for developers and users, internally Demos scans and selects from a list of bridges (on-chain) and ramps (off-chain via web2 endpoints) to identify the most efficient and secure route. This eliminates the need for the requester to manually select a bridge or ramp, thereby saving time and minimizing the exposure of funds. In addition to this efficient selection process, Demos incorporates a firewall layer that continuously and heuristically detects security issues on bridges and ramps as quickly as possible. The mechanism employed by the firewall operates in diverse ways, which will be explained in the following paragraph.

Demos Firewall

In the context of the Limited Liquidity Available scenario, as previously mentioned, Demos employs a sophisticated mechanism to identify and select the optimal bridge or ramp for providing a seamless, efficient, and expeditious experience to the requestor.

As widely acknowledged, bridges represent a prominent target for malicious actors seeking to exploit vulnerabilities and steal funds. To counteract this threat, Demos incorporates a firewall layer.

Constant Auditing

The Demos network employs a perpetual and automated process to audit the smart contracts and components of ramps and bridges, meticulously scrutinizing them for common vulnerabilities, exploits, and attacks. Despite variations in code implementation, it is noteworthy that a majority of bridge exploits stem from similar types of vulnerabilities. Demos vigilantly monitors the security posture of the platform to proactively thwart the utilization of insecure pathways, all while remaining imperceptible to the end user.

Immediate Response Propagation

In instances where a bridge or ramp falls victim to exploitation, the attackers' primary source of gain does not solely derive from the initial attack. The most substantial harm is inflicted upon users who, oblivious to the service's compromise, continue utilizing it for prolonged periods, sometimes spanning hours or even days, until they become aware of the situation.

Within this realm, Demos assumes the role of an all-encompassing, automated monitoring service. Even in cases where an exploit employs ingenious tactics that evade immediate detection, Demos leverages heuristic analysis of transactions, news reports, and official statements to swiftly identify the issue and impede vulnerable routes for all Demos users, without them noticing it.

Leveraging the aforementioned techniques, the Demos Firewall manifests as an advanced system that integrates heuristics, analytics, and machine learning, effectively safeguarding users, even when operating as an intelligent aggregator of bridges and ramps.

L2 Parallel Subnetwork (L2PS)

While the Demos Network is inherently fast and cheap, it still has to face some of the common problems that blockchain-like networks encounter in providing data.

Due to the transaction gas framework that is critical to maintain the network clean, safe and performant, some kinds of data exchange would require an impractical amount of gas and an unacceptable latency.

A classic example, which we will examine in detail later, is Instant Messaging. As of today, there are no fully on-chain mainstream solutions offering instant messaging in a practical form.

Before diving deep into this example, though, Demos L2PS has to be introduced.

What is a Subnetwork?

In the context of Demos Network specifications, a subnetwork is a network of Demos peers that in addition to the Demos main specifications integrate optional specifications defined in this paper.

The integrated and automatic subnetwork included in any Demos node client is the Instant Messaging Protocol (IMP) which is used intensively by the Demos Network to efficiently deliver communications without clogging the main chain.

Why Parallel?

Any L2PS integration in Demos must be at the very least fully compatible with a running Demos node and its pre built modules. This means that no L2PS subnetwork can be ran without a full Demos node installed and running.

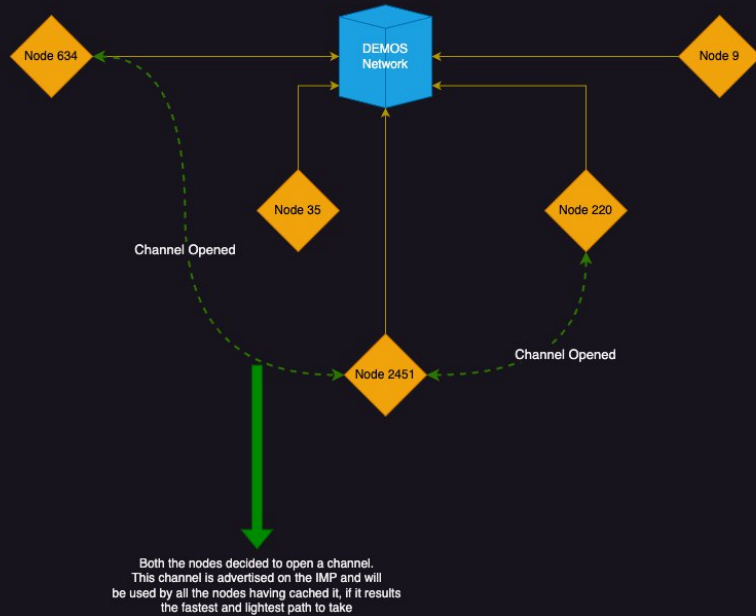
The “parallel” definition indicates that L2PS are not, as the usual L2 concept may hint, separate networks that catch up with an L1: they are, instead, parallel protocols and channels that are used to provide resources and services in the most efficient way as possible.

IMP: Demos built-in L2PS

Instant Messaging Protocol

To keep most of the load off-chains, DEMOS Network implements a P2P network on top of DEMOS called IMP (Instant Messaging Protocol).

By opening direct communication channels between themselves, enough nodes are able to keep track of a workflow (for example an instant messenger), witness each others and submit a proof of result on DEMOS once the flow has finished.



Practical Example

Bob wants to execute a given transaction (any valid DEMOS transaction). Its transaction instructs the RPC to use the IMP network when possible. All the workflow that is able to be channeled in the IMP will be executed, verified and computed off chain by using the same DEMOS trustless cryptography. Only the final, cryptographically secure result is then stored and attested on chain. The savings using IMP can reach 95% of the TX size.

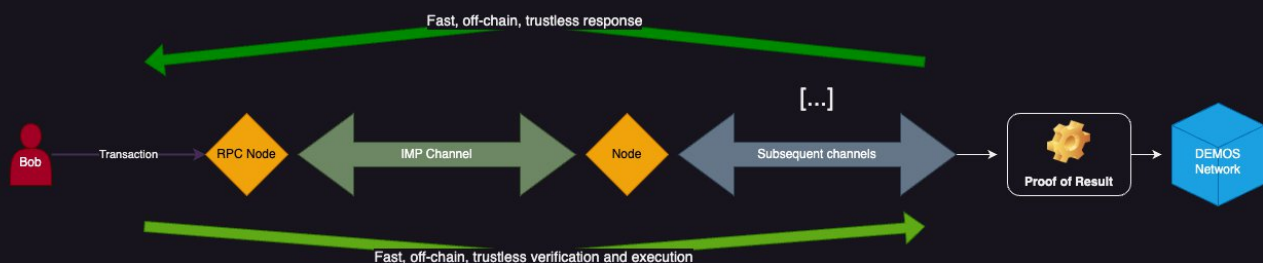


Figure 5: How IMP works

As mentioned above, Demos specifications define a mandatory L2PS called IMP. As Figure 5 suggests, IMP is designed to reduce network load while providing a proper and efficient communication channel that is usable in different ways.

IMP needs to create direct communication channels, as defined in Demos specifications, with near peers. While this is usually one of the weaknesses of an L2 network, in the L2PS scenario the IMP node is also the Demos node and thus inherits all of the peers from the beginning.

By building a dynamic, verifiable and unpredictable chain of IMP nodes, the L2PS is able to quickly and safely certify many types of data that don't require immediate finality or that don't require finality at all.

As usual, an example explains it better:

- Alice wants to start a chat session with Bob, and uses the Demos SDK to call the proper method
- The Demos SDK connects to the Demos RPC and sends the L2PS request
- The Demos RPC interprets the L2PS request and verify which operation is requested
- To find Bob, the Demos RPC uses its already established IMP network of peers to propagate the request "Where is Bob?"
- Due to the redundancy of the IMP connections, Bob is almost immediately found
- Alice's RPC either connects to Bob's current RPC (if Bob is connected and waiting for Alice), or writes the message in Bob's inbox
- Bob is able to either immediately receive or fetch the message
- Routine garbage management ensures the healthy condition of IMP L2PS

While this example explains in detail how L2PS logic works through an example of an IMP transaction, it also creates numerous questions that require a proper explanation. In the next paragraphs, we will examine each one of them.

Privacy and L2PS: A built-in concept

In the example above, the notable observation is the complete omission of any mention of privacy and sensitive information.

The Demos Network has a built in ZK Module (that is discussed in details in the proper chapter). This module is consistently updated, maintained, and applicable to various facets of Demos Transactions. Apart from the chat example which would be secure enough End-to-End encryption (also discussed in the next paragraph), there are many scenarios where an actor, to safeguard privacy, necessitates not only the temporary absence of data persistence but also the heightened privacy offered exclusively by Zero Knowledge technology.

The ZK (Zero Knowledge) Module

The Demos ZK Module is meticulously crafted from the ground up, utilizing battle-proven algorithms and comprising approximately 200 lines of code. It incorporates a comprehensive primer and a meticulous implementation of Magical Box logic to enable repeatable and consistent Zero Knowledge Proofing. This process involves a concealed "Toy" (the data) within a Magical Box, ensuring complete privacy by securing it with a Secret Key.

In the verification phase, the Verifier initiates a Coin Flip, receiving only the Magical Box. The Prover, tasked with demonstrating their knowledge of the box's content without revealing it, performs what is referred to as the Magical Trick. The successful execution of this Magical Trick is the ultimate outcome of the ZK Proofing process.

The Importance of a ZK Module in an Omnichain

The ZK module is crucial in an omnichain for several reasons:

1. **Privacy:** ZK proofs can prove the legitimacy of a transaction without revealing sensitive details from either party. This is particularly important in the context of blockchain technology, where transactions are typically public and traceable. By using ZK proofs, users can maintain their privacy while still participating in the blockchain ecosystem.
2. **Security:** ZK proofs provide a secure way to verify transactions. If a prover knows the secret, the ZK protocol should always succeed in convincing the verifier. Conversely, if a prover does not know the secret, the proof should always fail. This ensures the integrity of transactions and prevents fraudulent activity.
3. **Interoperability:** In an omnichain, multiple blockchains are interconnected. ZK proofs can be used to verify transactions across these different chains, enhancing the interoperability of the system.

Demos and Zero Knowledge

The ZK module enhances Demos by providing:

1. **Scalability:** ZK proofs help to minimize on-chain computation, thereby enhancing the scalability of the blockchain. This is because ZK proofs can validate a transaction without requiring the details of the transaction to be revealed or stored on the Demos blockchain.
2. **Efficiency:** ZK proofs are succinct, meaning they can be quickly verified, which is crucial for maintaining the efficiency of the Demos Network and lowering the entry barrier.
3. **Trustless Environment:** In a decentralized system like an omnichain, trust is a significant concern. ZK proofs enable a trustless environment where parties can verify each other's claims without needing to trust each other.

In the context of a Demos powered omniweb, the ZK module is a critical component, providing privacy, security, interoperability, scalability, and efficiency on demand while not being enforced if not requested. This enables a trustless environment where transactions can be verified without revealing sensitive information, significantly expanding Demos scalability and versatility.

Demos ZK Module and beyond

The Demos Network ZK Module is equipped with a comprehensive interactive Zero Knowledge proof suite, which incorporates the Miller-Rabin Test for generating large prime numbers.

The Miller-Rabin Test is a probabilistic algorithm used to determine whether a given number is prime. It is particularly useful in cryptographic applications where large primes are essential for security, such as in the generation of RSA keys. By repeatedly testing random bases against the number in question, the Miller-Rabin Test can provide a high level of confidence in the primality of the number, making it a valuable tool for Zero Knowledge proofs where the hardness of certain mathematical problems, like factoring large primes, underpins security.

The Demos Network further extends the utility of its ZK Module by offering Prover and Verifier classes. These classes simplify the process of generating commitments and responding to challenges, thereby streamlining the creation and verification of Zero Knowledge proofs. This simplification is crucial for developers and users who require a standardized and accessible way to engage with cryptographic protocols.

In addition to its core offerings, the Demos Network facilitates the integration of custom ZK services. This means that partners and infrastructures can seamlessly incorporate their specialized ZK services, enhancing the network's capabilities. For example, a partner could integrate a hardware-based ZK computation service that accelerates the proof generation process, similar to how certain startups are developing FPGA and ASIC solutions to speed up cryptographic computations.

Another potential integration could involve a cloud-based ZK service that offers scalable proof generation through a SaaS model, providing API access to clients who require on-demand cryptographic services without the need for in-house infrastructure.

These custom integrations exemplify the flexibility of the Demos Network, allowing it to cater to a diverse range of cryptographic needs and use cases. By supporting such integrations, the network not only broadens its applicability but also fosters an ecosystem where innovation in Zero Knowledge proof generation can thrive, ultimately contributing to a more secure and efficient blockchain landscape.

Homomorphic Encryption: Tailoring Privacy for Real Use Cases

To establish a robust privacy framework, Demos acknowledges the necessity of a novel approach to Zero-Knowledge operations, complementing traditional ZK Proofs. Consider a scenario where a user wishes to transmit encrypted data without disclosing it, yet enabling others to perform trusted and verifiable operations on the data.

This approach is known as Homomorphism, and it involves the use of Homomorphic Encryption for secure data manipulation. Specifically, Fully Homomorphic Encryption (FHE) empowers Demos users and developers to create cutting-edge, privacy-preserving applications on the Demos Network. FHE ensures that users can enjoy the benefits of advanced privacy features without compromising on dynamism and functionality.

A Practical Example of FHE

Given the recent developments in Fully Homomorphic Encryption (FHE) and Homomorphic Encryption in general, it's helpful to explore a potential use case. Consider the scenario where Alice possesses a confidential number and wants Bob to add his data to this number without revealing the specifics.

Traditionally, Alice might ask Bob for the sum, verify its authenticity, manually add it, and then provide Bob with a proof of the completed operation. However, with an FHE approach, these steps can be streamlined into distributed processes:

- Alice encrypts her sensitive variable containing the number 5 using Homomorphic Encryption within the Demos SDK
- Bob, receiving the encrypted message, is unable to decipher its content
- Using the Demos SDK's FHE module, Bob adds his value (2) to the encrypted message without needing to decrypt it
- Alice, receiving the modified and encrypted data, decrypts it using the Demos SDK, as she originally encrypted it
- The decrypted data reads 7 ($5+2$), and Alice can cryptographically verify the result with security measures in place

A Trustless Approach to Data Transformation

The simplified example above serves as an illustration of how privacy is intricately woven into the user experience within the Demos Network specifications. Emphasizing the significance of employing suitable technology for specific features, Fully Homomorphic Encryption (FHE) introduces a rapid and focused layer of security across the entire network.